

Posterior Sampling

Frank Edwards

Sampling from the posterior

- We use Bayesian methods to *approximate* the posterior distribution
- We can then sample parameters from the estimated posterior to learn about parameters or to simulate predictions
- We can plot densities, directly sample *credible intervals*, compare differences in means, all without relying on a theoretical *sampling distribution*, like we do with frequentist methods

Summarizing distributions

Let's say we want to know how much probability mass there is within ± 1.4 standard deviations of 0 for $x \sim N(0, 1)$

The Normal probability density function (PDF) is:

$$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{x-\mu}{\sigma} \right)^2}$$

Integrals are hard

To answer our question exactly, we can take the integral of the PDF from -1.4 to 1.4 :

$$\int_{-1.4}^{1.4} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} dx$$

- I don't want to do this, and I suspect neither do you.
- *Note:* We can just use `dnorm()` here, but most of our posteriors won't be so simple to work with.

But simulation is easy!

- . Sample from the target distribution

```
1 n_samples<-10000  
2 samples<-rnorm(n = n_samples, mean = 0, sd = 1)
```

- . Summarize our quantity of interest

```
1 ggplot(data.frame(samples), aes(x=samples)) + geom_histogram()
```

Using samples to calculate quantities of interest

- What are the mean and SD of this parameter?

```
1 mean(samples)
```

```
[1] -1.674023e-05
```

```
1 sd(samples)
```

```
[1] 0.9996785
```

- How much mass is within $0 \pm 1.4\sigma$

```
1 samples_df<-data.frame(samples)
2 samples_df |>
3   filter(samples>=-1.4 & samples<=1.4) |>
4   summarise(mass = n()/n_samples)
```

```
      mass
1 0.8377
```

Estimation strategies

- For products of probability distributions (ie $prior \times likelihood$); there usually aren't exact solutions.
- We'll rely on 3 algorithms to *approximate* posterior distributions to condition the prior on the likelihood of the data.

Our approximation engines

- Grid approximation (check!)
- Quadratic approximation (today!)
- Markov Chain Monte Carlo (MCMC) (week 9 on, with some in week 5/6)

Grid approximation algorithm

- . Define the grid
- . Compute the prior for each parameter value on the grid
- . Compute the likelihood for each parameter value on the grid
- . Multiply the prior by the likelihood
- . Divide by the sum of all values

Grid approximation in R

We draw (with replacement) six green marbles from a bag that we know has red and green marbles; we want to know the proportion green (p) in the bag.

```
1 ### size of grid
2 length <- 50
3 ### make our grid
4 grid<-seq(from = 0, to = 1, length.out = length)
5 ### uniform (flat) prior
6 prior <- rep(1, length)
7 ### likelihood, observe 6 of 9 green, how likely under each value of p?
8 likelihood <- dbinom(6, size = 9, prob = grid)
9 ### update the posterior
10 posterior <- prior * likelihood / sum(prior * likelihood)
```

Sampling from a grid-estimated posterior

- We've estimated the posterior probability density for each value on the grid
- We can draw random samples from this distribution.
- With a large enough number of samples, we can closely approximate the distribution
- Using these samples, we can learn a lot about the parameter of interest

Draw samples of proportion water from grid estimated posterior

```
1 post_samples<-sample(  
2   grid,  
3   prob = posterior,  
4   size = 10,  
5   replace = T)  
6  
7 round(post_samples,2)
```

```
[1] 0.67 0.65 0.55 0.55 0.47 0.73 0.57 0.27 0.69 0.43
```

Our estimate of p , 10 posterior draws

```
1 plot_dat<-data.frame(  
2   post_samples = sample(  
3     grid,  
4     prob = posterior,  
5     size = 10,  
6     replace = T))  
7  
8 ggplot(plot_dat,  
9   aes(x = post_samples)) +  
10  geom_density() +  
11  coord_cartesian(xlim=c(0,1))
```

100 posterior draws

```
1 plot_dat<-data.frame(  
2   post_samples = sample(  
3     grid,  
4     prob = posterior,  
5     size = 100,  
6     replace = T))  
7  
8 ggplot(plot_dat,  
9   aes(x = post_samples)) +  
10  geom_density() +  
11  coord_cartesian(xlim=c(0,1))
```

1000 draws

```
1 plot_dat<-data.frame(  
2   post_samples = sample(  
3     grid,  
4     prob = posterior,  
5     size = 1000,  
6     replace = T))  
7  
8 ggplot(plot_dat,  
9   aes(x = post_samples)) +  
10  geom_density() +  
11  coord_cartesian(xlim=c(0,1))
```

10000 draws

```
1 plot_dat<-data.frame(  
2   post_samples = sample(  
3     grid,  
4     prob = posterior,  
5     size = 10000,  
6     replace = T))  
7  
8 ggplot(plot_dat,  
9   aes(x = post_samples)) +  
10  geom_density() +  
11  coord_cartesian(xlim=c(0,1))
```

100000 draws

```
1 plot_dat<-data.frame(  
2   post_samples = sample(  
3     grid,  
4     prob = posterior,  
5     size = 100000,  
6     replace = T))  
7  
8 ggplot(plot_dat,  
9   aes(x = post_samples)) +  
10  geom_density() +  
11  coord_cartesian(xlim=c(0,1))
```

So what now?

What questions do we typically ask about a parameter?

- What is the mean / expected value of the parameter?
- How certain are we about the location of the parameter?

Using posterior samples

- What is $E(p)$

```
1 mean(plot_dat$post_samples)
```

```
[1] 0.6367445
```

- How certain are we about the location of p ? Where is 92 percent of the posterior probability?

```
1 quantile(plot_dat$post_samples, c(0.04, 0.96))
```

```
      4%      96%  
0.3877551 0.8571429
```

- Conditional on the data and our assumptions (priors, likelihood), we can describe the posterior *without* appealing to theoretical replications (!!!)

Exact binomial likelihood

Posterior approximation

Quadratic approximation

- . Assume our posterior is approximately Normal (often reasonable)
- . Find the mode
- . Estimate curvature of the log posterior with a parabola
- . Use the resulting Gaussian distribution for inference
- . With enough data: quap $\rightarrow$ MLE

Using quadratic approximation in R

```
1 ### First, define the data as a list() object
2 ### let's treat 1 as green, 0 as red
3 dat<-list(G = 6, R = 3)
```

Using quadratic approximation in R

```
1 ## Specify the prior and likelihood in alist() object
2 formula_list<-alist(
3   # likelihood
4   G ~ dbinom(R+G, p),
5   # prior
6   p ~ dunif(0,1)
7 )
```

Using quadratic approximation in R

```
1 model1<-quap(  
2   formula_list,  
3   data = dat  
4 )
```

Using quadratic approximation in R

```
1 # summarize the distribution using summary() or precis()  
2 summary(model1)
```

```
      mean      sd      5.5%      94.5%  
p 0.6666667 0.1571338 0.4155366 0.9177968
```

Extract posterior samples

```
1 post_samples<-extract.samples(model1, n=10000)
```

Visualize

```
1 p1<-ggplot(post_samples, aes(y = p, x = 1:nrow(post_samples))) +  
2   geom_point(alpha = 0.1)  
3 p2<-ggplot(post_samples, aes(x = p)) +  
4   geom_density()  
5 ### grid.arrange() lets us put more than one plot in a window  
6 grid.arrange(p1, p2, ncol = 2)
```

Using this object

```
1 ### peek inside
2 head(post_samples)
```

```
      p
1 0.6369756
2 0.5745009
3 0.6673372
4 0.5324838
5 0.9078291
6 0.6811012
```

Learning from the posterior

Questions we can ask with posterior samples:

- How much posterior probability lies (below/above/between) some parameter value(s)?
- Which parameter values mark the (lower/upper) $n\%$ of the posterior probability?
- Which parameter value has the highest posterior probability?

Probability above/below

- We can simply filter the samples, then divide by the size of the posterior sample.
- **Q:** What is the posterior probability that the bag is less than 60 percent green, conditional on our model and the data (6G, 3R)?

```
1 post_samples |>
2   filter(p<0.6) |>
3   summarise(p = n()/nrow(post_samples))
```

```
      p
1 0.3358
```

Probability above/below

- **Q:** What is the posterior probability that there is between 60 percent and 90 percent green marbles in the bag, conditional on our model and the data?

```
1 post_samples |>
2   filter(p>=0.6 & p<=0.9) |>
3   summarise(p = n()/nrow(post_samples))
```

```
      p
1 0.599
```

Confidence (posterior) intervals

- Q: Where does 90% of the posterior probability mass lie?

```
1 quantile(post_samples$p, c(0.05, 0.95))
```

```
      5%      95%  
0.4062213 0.9219134
```

```
1 quantile(post_samples$p, 0.9)
```

```
      90%  
0.8641941
```

- Thoughts on the differences between these two intervals?

Posterior intervals

Confidence intervals versus Bayesian intervals

- Confidence interval: If we replicated the experiment many times, the true value would lie within this interval 90 percent of the time.
- Posterior interval: Conditional on our data and model (priors, likelihood), 90 percent of compatible parameter values lie within this range.

Point estimates

- Which parameter value has the highest posterior probability?

```
1 chainmode(post_samples$p, adj = 0.01)
```

```
[1] 0.7268666
```

- This is the maximum a posteriori (MAP) estimate.
- If our posterior is approximately symmetric, the MAP and median will be similar

```
1 median(post_samples$p)
```

```
[1] 0.6668776
```

Simulating prediction

- We've learned how to describe the parameter p with posterior samples.
- What would happen if we re-ran the experiment?
- **Posterior predictive simulation** incorporates updated information on the parameter into predictions of new values of our observed outcome variable (W, L)

Posterior prediction algorithm

- . Sample from the posterior distribution
- . Use these sampled posterior parameter values to sample new values from the likelihood

Posterior prediction in R

```
1 # Draw samples of posterior p from model1
2 # Truncate impossible  $p > 1$  (Gaussian has long tails)
3 p_samples<-extract.samples(model1) |>
4   mutate(p = ifelse(p>1, 1, p))
5 ## sample from likelihood
6 RG_predictions<-rbinom(1e4, size = 9, prob = p_samples$p)
7 ## distribution of predictions of G under our posterior
8 table(RG_predictions)
```

```
RG_predictions
  0    1    2    3    4    5    6    7    8    9
26  125  312  670 1123 1589 1817 1880 1439 1019
```

Visualize the predictions

```
1 ggplot(data.frame(RG_predictions),  
2       aes(x = RG_predictions)) +  
3   geom_histogram() +  
4   scale_x_continuous(breaks=0:9) +  
5   labs(title = "Posterior predictions of W")
```

The impact of uncertainty in p on predictions

```
1 ggplot(p_samples, aes(x = p)) +  
2   geom_histogram()
```

p = posterior median

```
1 ## set p at the median of the posterior
2 samples<-rbinom(1e4, 9, median(p_samples$p))
3 ggplot(data.frame(samples), aes(x = samples)) +
4   geom_histogram()
```

p = posterior 5th percentile

```
1 ## set p at the 5th percentile of the posterior
2 samples<-rbinom(1e4, 9, quantile(p_samples$p, 0.05))
3 ggplot(data.frame(samples), aes(x = samples)) +
4   geom_histogram()
```

p = posterior 95th percentile

```
1 ## set p at the 95th percentile of the posterior
2 samples<-rbinom(1e4, 9, quantile(p_samples$p, 0.95))
3 ggplot(data.frame(samples), aes(x = samples)) +
4   geom_histogram()
```

Propagating ALL uncertainty

Posterior predictive distributions propagate uncertainty in p into predictions

```
1 ggplot(data.frame(RG_predictions),  
2         aes(x = RG_predictions)) +  
3     geom_histogram() +  
4     scale_x_continuous(breaks=0:9) +  
5     labs(title = "Posterior predictions of G")
```

Prior prediction

We can also use simulation to better understand and select priors

```
1 ### our prior
2 formula_list
```

```
[[1]]
G ~ dbinom(R + G, p)
```

```
[[2]]
p ~ dunif(0, 1)
```

```
1 prior_samples<-runif(1e4, 0, 1)
2
3 head(prior_samples)
```

```
[1] 0.5836418 0.2305983 0.5799636 0.7134089 0.8438489 0.3605211
```

Visualize the prior predictions

```
1 ggplot(data.frame(prior_samples),  
2       aes(x = prior_samples)) +  
3   coord_cartesian(xlim = c(0, 1)) +  
4   geom_histogram(bins = 20)
```

Summary

- Simulation is a fundamental tool in Bayesian data analysis
- Visualize samples from the prior, posterior, and posterior predictive distributions
- Posterior prediction far more informative than point estimate / std. error approach

Speaker notes